

Exam Analytics Pro: A Hybrid ML–Agentic System for Exam Question Difficulty Prediction

Progyan Sen, Roni Maity, Arpit Sarang, Ganesh Wayal
Team: prag

Abstract—We present *Exam Analytics Pro*, a hybrid system for predicting exam question difficulty, combining TF-IDF-based Logistic Regression with an agentic LangGraph pipeline powered by frontier LLMs. Trained on 300,000 StackOverflow questions (Easy / Medium / Hard), Logistic Regression achieves 55.62% accuracy and macro F1 of 0.55—well above the 33.3% random baseline—outperforming a Decision Tree (46.81%). The agentic layer grounds verdicts in Bloom’s Taxonomy for explainability, surfaced via a public Streamlit dashboard.

Index Terms—Educational analytics, TF-IDF, Logistic Regression, LangGraph, Bloom’s Taxonomy, explainable AI.

I. INTRODUCTION

Calibrating exam question difficulty manually is subjective and unscalable across large question banks. Automated prediction offers a principled alternative, yet most systems produce opaque labels with no pedagogical justification. We propose a two-stage system: (1) a TF-IDF + Logistic Regression classifier for fast baseline labelling, and (2) an LLM-based agentic layer that maps questions to Bloom’s Taxonomy [?] and explains verdicts, deployed in a public Streamlit dashboard.

II. RELATED WORK

Qiu et al. [?] applied TF-IDF with logistic regression to programming question difficulty on online judges, demonstrating competitive NLP baselines. Lee and Kim [?] extended this with BERT embeddings, improving accuracy at the cost of interpretability. LangChain [?] and LangGraph [?] have enabled stateful multi-step LLM pipelines; our work is among the first to ground such reasoning in a prior ML verdict for pedagogical tasks.

III. METHODOLOGY

A. Dataset Description

The StackOverflow Kaggle dump (1,000,000 questions (raw dataset)) is filtered to retain posts with at least one answer, view count > 50, and non-null text. A stratified sample of 300,000 records is drawn. Difficulty is derived from engagement:

$$\text{Difficulty} = \frac{\text{AnswerCount}}{\text{ViewCount}} \quad (1)$$

Quantile thresholds (33%, 66%) yield three balanced classes: Easy (101,575), Medium (99,650), Hard (98,775).

B. Preprocessing

Raw text is cleaned via BeautifulSoup (HTML/code-block removal), regex URL stripping, and whitespace normalisation (src/data_prep.py). No imputation is required as filtered records are non-null by construction.

C. Feature Engineering

Two feature blocks are assembled and concatenated via `scipy.sparse.hstack` into a $(299,997 \times 50,002)$ matrix: (a) **TF-IDF**—unigrams + bigrams, 50,000 vocabulary, `min_df=5`, capturing pedagogical cues (e.g., *calculate*, *evaluate*); (b) **Numeric**—question character length and community score, scaled with `StandardScaler`.

D. Models Used

Logistic Regression (solver `saga`, `max_iter=5000`, `class_weight=balanced`) is the primary classifier; its label output feeds the agentic layer via `get_ml_prediction()` in `src/ai_engine.py`. **Decision Tree** serves as an interpretable baseline, with branch paths surfaced in the dashboard. Both models are serialised as `lr_model.joblib`, `dt_model.joblib`, and `vectorizer.joblib`.

E. Training and Validation

An 80/20 stratified train-test split is applied (240,000 train / 60,000 test). Performance is measured by accuracy, per-class precision, recall, F1-score, and macro-averaged F1. The `class_weight=balanced` setting prevents the optimiser from favouring any class.

IV. RESULTS

A. Quantitative Results

TABLE I
MODEL COMPARISON — HELD-OUT TEST SET (60,000 SAMPLES)

Model	Acc	Easy F1	Med F1	Hard F1
Logistic Reg.	0.5562	0.61	0.48	0.57
Decision Tree	0.4681	0.50	0.44	0.46
Random Base.	0.3333	—	—	—

LR outperforms DT by 8.81 percentage points and both exceed random chance by >13 pp. Medium is the weakest class for both models (LR F1: 0.48; DT F1: 0.44) due to soft quantile boundaries. Confusion matrices confirm the DT over-misclassifies Easy→Hard (7,642 vs. LR’s 4,360), explaining the larger accuracy gap.

TABLE II
LR CONFUSION MATRIX. ROWS: TRUE; COLS: PREDICTED.

	Easy	Medium	Hard
Easy	13,370	2,546	4,360
Medium	4,240	10,903	4,668
Hard	6,145	4,668	9,099

TABLE III
DT CONFUSION MATRIX. ROWS: TRUE; COLS: PREDICTED.

	Easy	Medium	Hard
Easy	11,004	1,630	7,642
Medium	5,492	7,177	7,142
Hard	7,420	2,589	9,903

B. Figures

V. DISCUSSION

The engagement proxy (Eq. 1) introduces label noise—niche questions score low due to small audiences, not inherent difficulty, creating the observed Medium boundary ambiguity. TF-IDF captures surface-level pedagogical cues (e.g., action verbs) but loses word order and context, imposing a ceiling on accuracy. The LangGraph agentic layer mitigates this by reasoning over question semantics via Bloom’s Taxonomy, providing educators with interpretable verdicts beyond a bare label. A key limitation is domain specificity: models trained on software Q&A may not generalise to mathematics or biology without retraining.

VI. CONCLUSION

Exam Analytics Pro demonstrates that combining TF-IDF statistical inference with LLM-based agentic reasoning yields a transparent, educator-ready difficulty assessment tool. Logistic Regression achieves 55.62% accuracy on a balanced three-class task, and the LangGraph pipeline bridges the gap between black-box ML and explainable pedagogy. Future directions include contextual embeddings, expert-annotated ground truth, and multi-domain adaptation.

REFERENCES

APPENDIX

This appendix provides details regarding the project’s GitHub repository for reproducibility and open-source collaboration.

A. Repository Link

The complete source code, trained models, and documentation are available at:

<https://github.com/praggCode/Exam-Analytics-System>

Fig. 1. Predicted class distribution across Easy, Medium, and Hard categories for an uploaded question batch.

B. Repository Structure

```
Exam-Analytics-System/
|-- data/
|   |-- posts_questions.csv
|   |-- processed/processed_data.csv
|-- models/
|   |-- lr_model.joblib
|   |-- dt_model.joblib
|   |-- vectorizer.joblib
|-- src/
|   |-- data_prep.py
|   |-- data_peek.py
|   |-- model_train.py
|   |-- ai_engine.py
|-- ui/app.py
|-- requirements.txt
|-- README.md
```

C. Description of Key Components

- **src/data_prep.py:** HTML cleaning, URL removal, difficulty labelling via Eq. 1, outputs `processed_data.csv`.
- **src/model_train.py:** Fits TF-IDF vectoriser and trains LR + DT; serialises all `.joblib` artefacts.
- **src/ai_engine.py:** LangGraph state machine; `get_ml_prediction()` returns a label string forwarded to the LLM node.
- **src/data_peek.py:** Exploratory utilities—class counts, length histograms, sample inspection.
- **ui/app.py:** Streamlit entry point; CSV upload, prediction pipeline invocation, visual dashboards.
- **models/:** `lr_model.joblib`, `dt_model.joblib`, `vectorizer.joblib`.
- **requirements.txt:** Pinned dependencies (`scikit-learn`, `langchain`, `langgraph`, `streamlit`, `beautifulsoup4`, LLM SDKs).
- **README.md:** Setup, training, and dashboard launch instructions.